Práctica Nro. 2

Programación con Pthreads

Información útil para compilar y ejecutar:

- Para compilar en Linux con gcc+Pthreads: *gcc -pthread –o salidaEjecutable archivoFuente*
- Para ejecutar: *./salidaEjecutable arg1 arg2 … argN*

Pautas generales

- Para obtener el tiempo de ejecución de todos los algoritmos se debe utilizar la función provista por la cátedra (*dwalltime*).
- Por convención sólo deberá tomarse el tiempo de ejecución del procesamiento de datos. Esto significa excluir del tiempo de ejecución:
  - *Reserva y liberación de memoria.*
  - *Inicialización de estructuras de datos.*
  - *Impresión y verificación de resultados.*
  - *Impresión en pantalla (printf)*
- Tanto en su máquina personal como en el equipo remoto de la cátedra, las pruebas deben realizarse de forma aislada a la ejecución de otras aplicaciones. En su máquina, se debe ejecutar desde consola, sin otras aplicaciones ejecutándose al mismo tiempo. En el equipo remoto, siga las indicaciones del instructivo.
- Para todos los ejercicios, considere lo siguiente al momento de desarrollar código:
  - ¿Qué tipo de paralelismo le parece más adecuado para este caso? ¿Funcional o de datos?
  - Analice cómo distribuir el trabajo entre los hilos. ¿Qué técnica de descomposición aplicaría para obtener las tareas?
  - Identifique las regiones de código que pueden ejecutarse en paralelo y las que no. ¿Existen dependencias? ¿Cómo sería el grafo de dependencias de tareas?
- Para todos los ejercicios, considere lo siguiente al momento de analizar el rendimiento:
  - ¿Qué algoritmo secuencial conviene utilizar?
  - ¿Cómo debe presentar tablas y gráficos de métricas?
  - ¿Qué puede decir sobre el rendimiento de su algoritmo ante cambios en T y N?
  - ¿Qué puede decir sobre escalabilidad?

Ejercicios

1. Desarrolle un algoritmo paralelo que compute una suma de vectores: $A_i = B_i + C_i$. Para ello, siga las pautas indicadas más arriba. Mida el tiempo de ejecución para distintos valores de N y T = {2, 4, 8}, y luego analice el rendimiento.

2. Desarrolle un algoritmo paralelo que compute la multiplicación de matrices cuadradas de *NxN* a partir de la versión optimizada del ejercicio 6 de la práctica anterior. Mida el tiempo de ejecución para N={512, 1024, 2048, 4096} y T={2,4,8}, y luego analice el rendimiento.

3. Desarrolle un algoritmo paralelo que compute la multiplicación de matrices cuadradas de *NxN* a partir de la versión que computa por bloques. Mida el tiempo de ejecución para N={512, 1024, 2048, 4096} y T={2,4,8}, y luego analice el rendimiento. Nota: puede asumir que el tamaño de bloque (TB) es múltiplo de *N*.

4. Desarrolle un algoritmo paralelo que cuente la cantidad de veces que un elemento X aparece dentro de un vector de N elementos enteros. Al finalizar, la cantidad de ocurrencias del elemento X debe quedar en una variable llamada *ocurrencias*. Para la sincronización emplee mutex-locks. Pruebe con diversos tamaños de N y T={2,4,8}. Analice el rendimiento.

5. Desarrolle un algoritmo paralelo que calcule el valor promedio, mínimo y máximo de los números almacenados en un vector de tamaño N. Para la sincronización emplee semáforos. Pruebe con diversos tamaños de N y T={2,4,8}. Analice el rendimiento.

6. Si tuviese que resolver el ejercicio 1 en el caso de una arquitectura heterogénea, como los nuevos procesadores híbridos de Intel[1], ¿utilizaría la misma estrategia de distribución de trabajo o una diferente? ¿Qué se debe tener en cuenta en este caso?

[1] https://www.intel.com/content/www/us/en/support/articles/000091896/processors.html